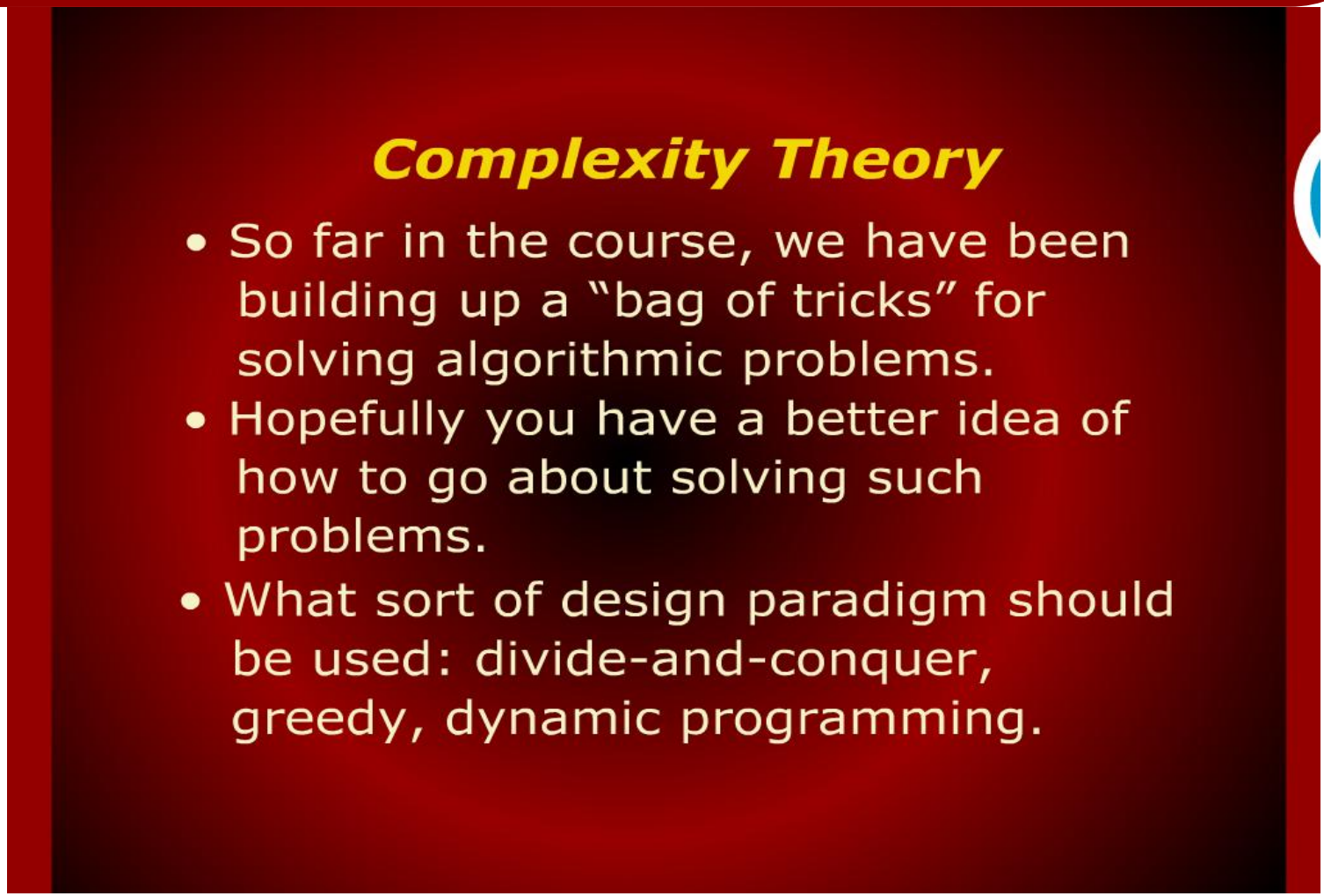
Algorithms

Lecture #43

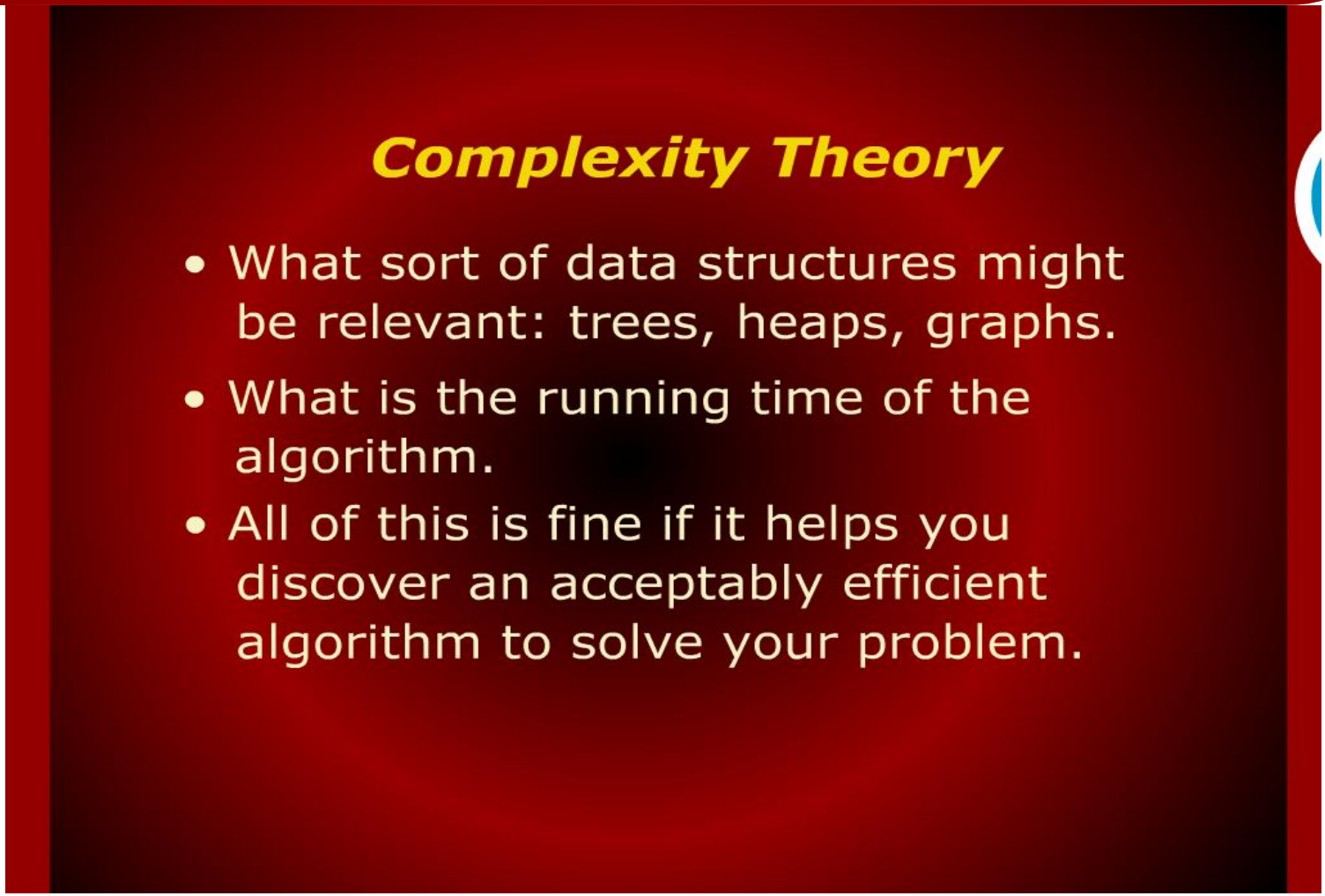
Syed Hamed Raza

Complexity Theory

Complexity Theory



Complexity Theory

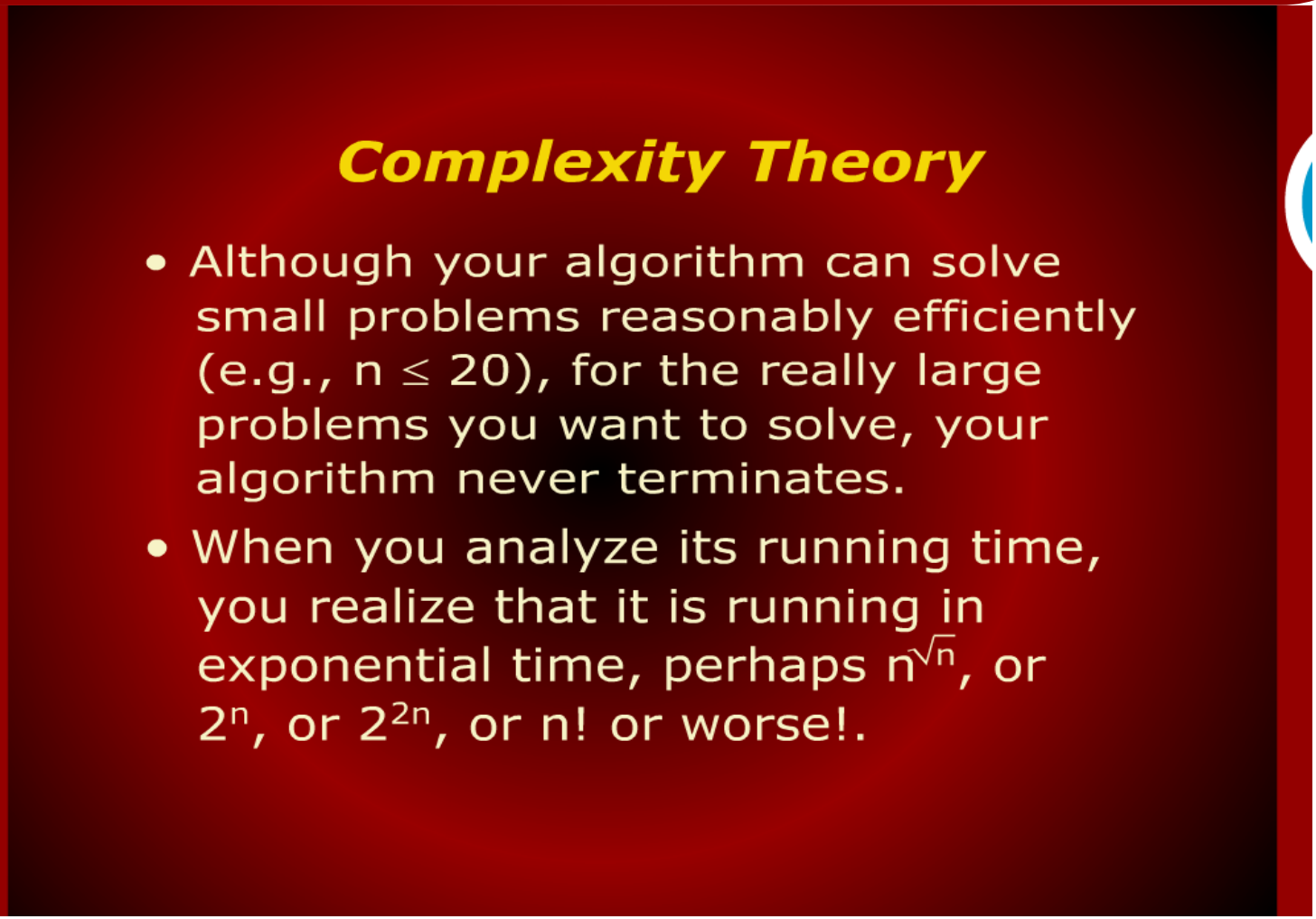


Complexity Theory

Complexity Theory

- The question that often arises in practice is that you have tried every trick in the book and nothing seems to work.
- Although your algorithm can solve small problems reasonably efficiently (e.g., $n \leq 20$), for the really large problems you want to solve, your algorithm never terminates.

Complexity Theory

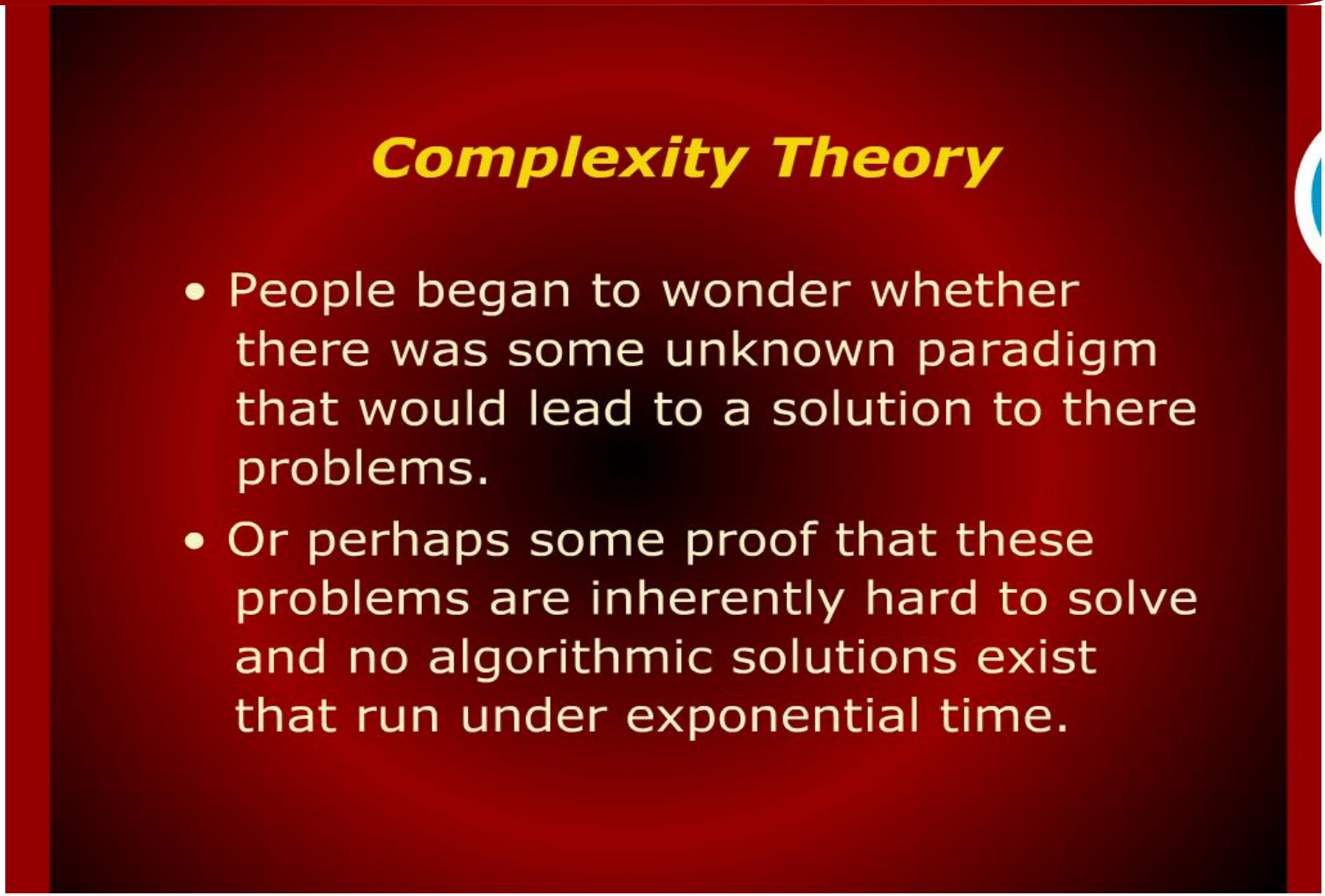


Complexity Theory

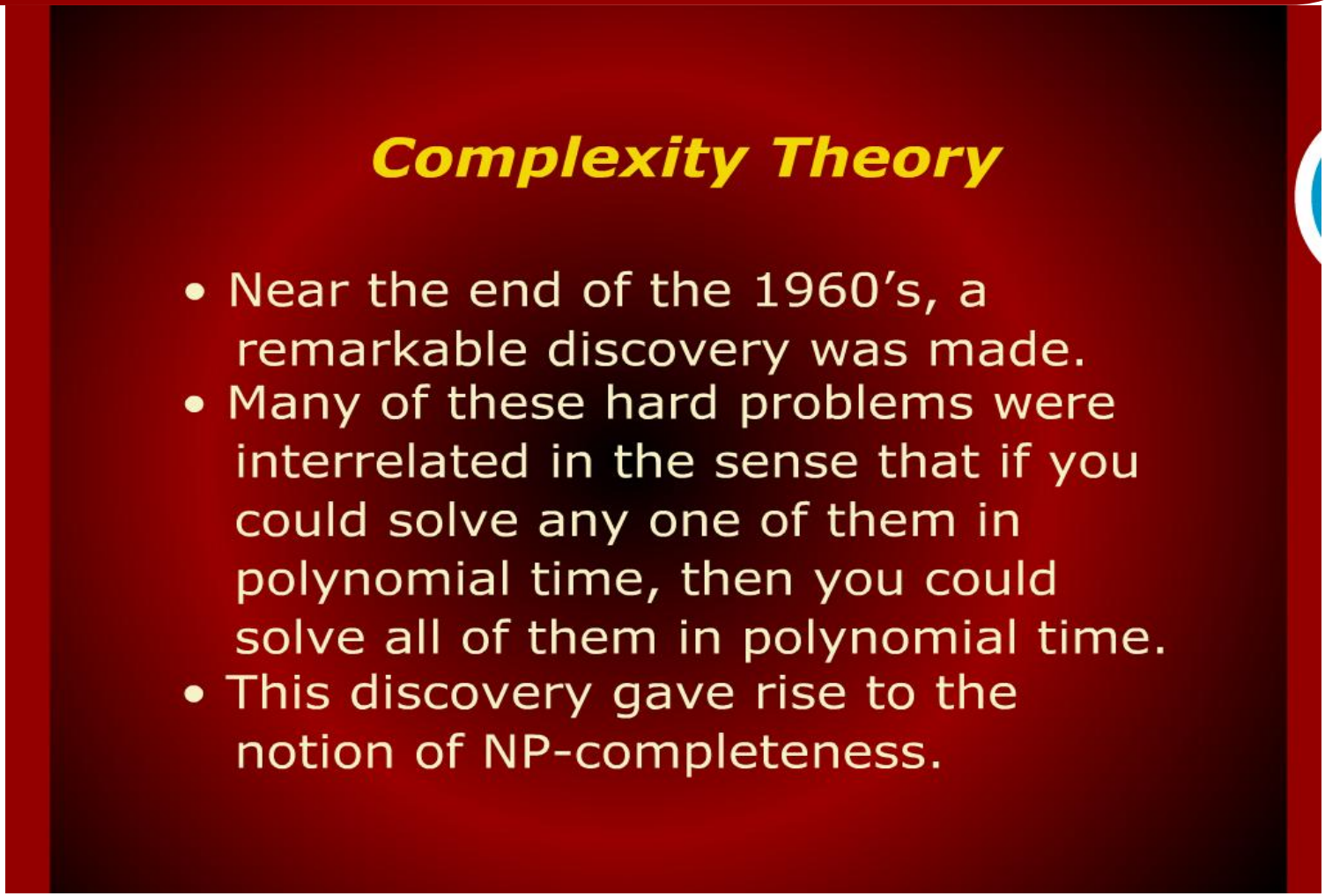
Complexity Theory

- By the end of 60's, there was great success in finding efficient solutions to many combinatorial problems.
- But there was also a growing list of problems for which there seemed to be no known efficient algorithmic solutions.

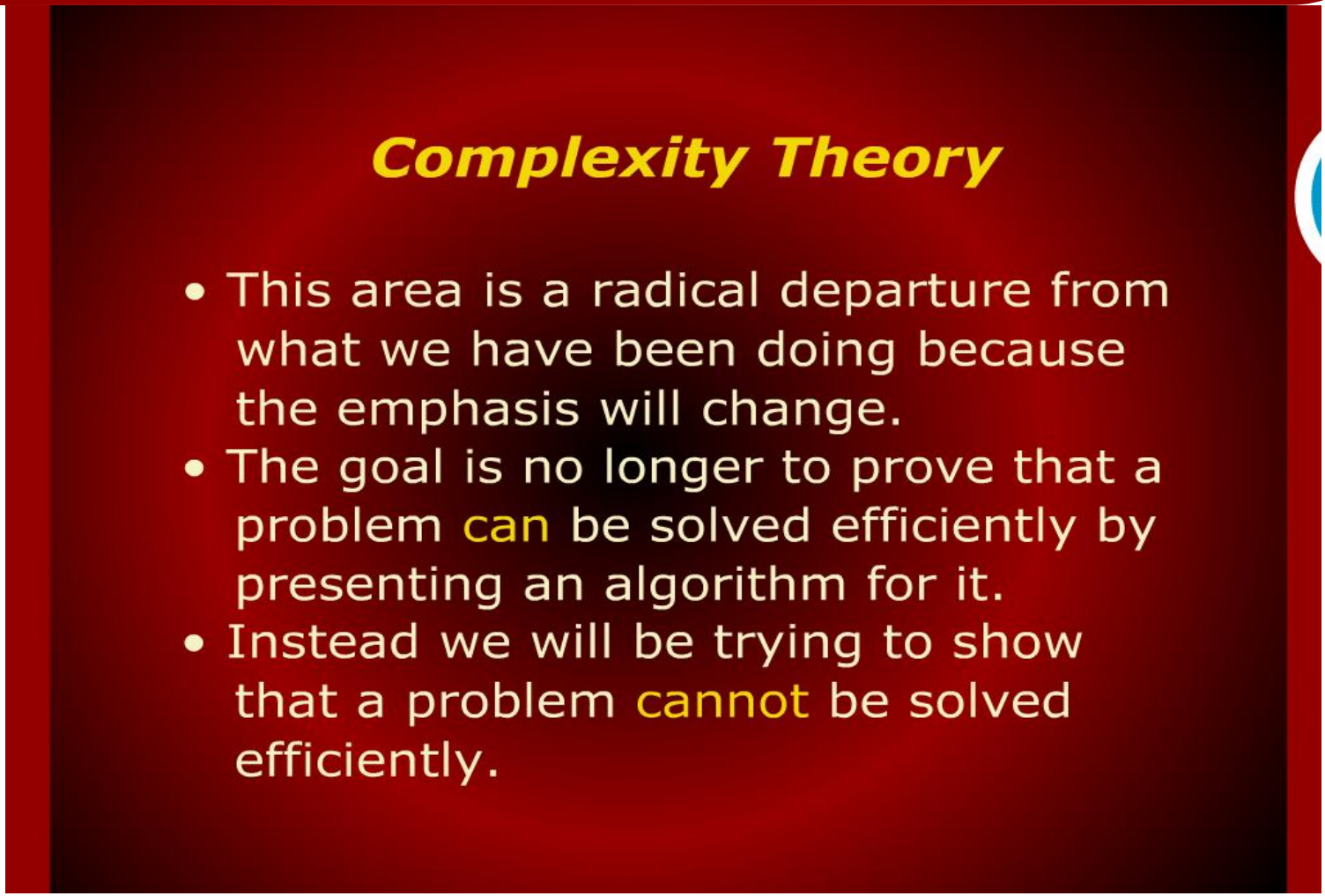
Complexity Theory



Complexity Theory



Complexity Theory



Complexity Theory

Complexity Theory

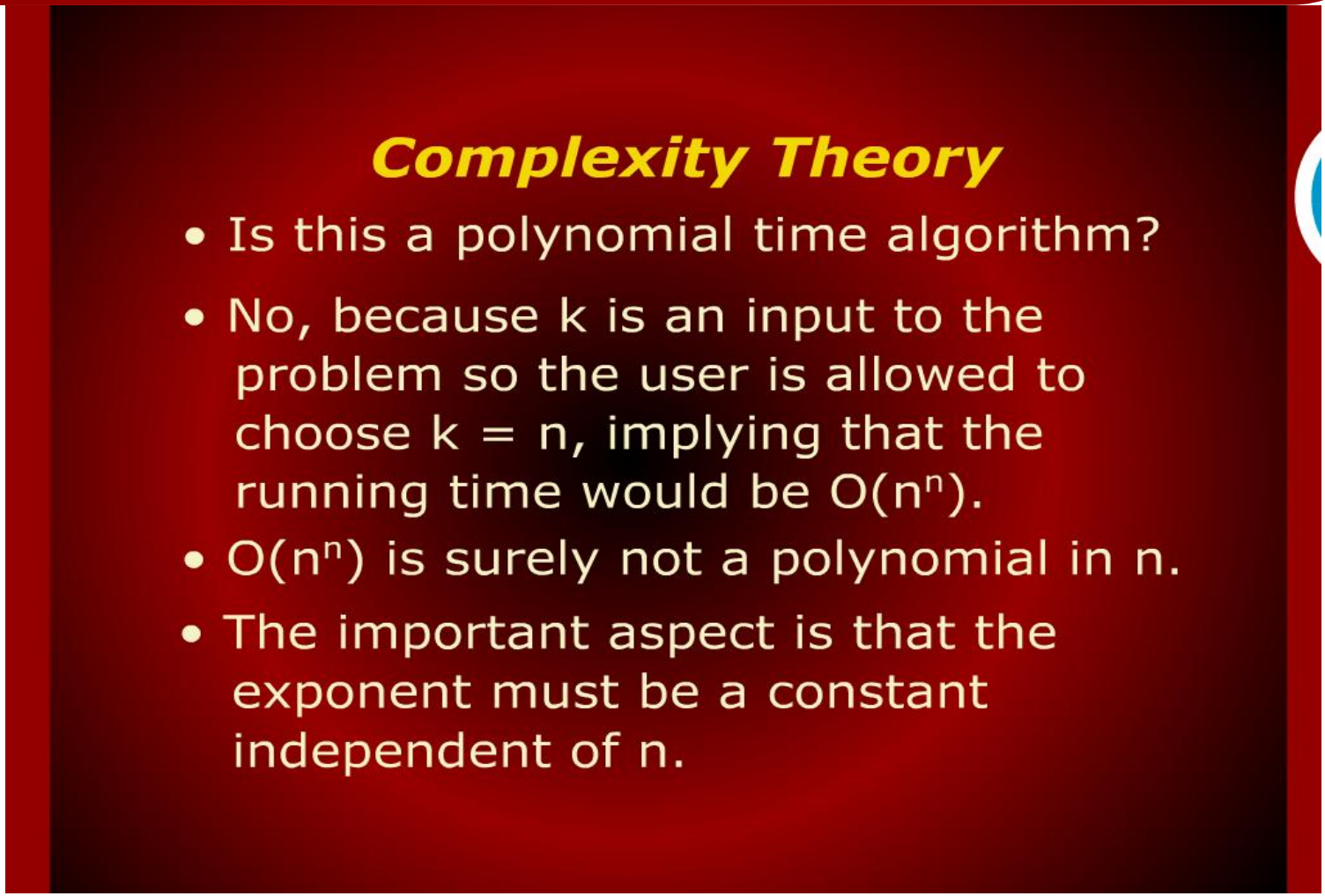
- Up until now all algorithms we have seen had the property that their worst-case running time are bounded above by some polynomial in n.
- A polynomial time algorithm is any algorithm that runs in $O(n^k)$ time.
- A problem is solvable in polynomial time if there is a polynomial time algorithm for it.

Complexity Theory

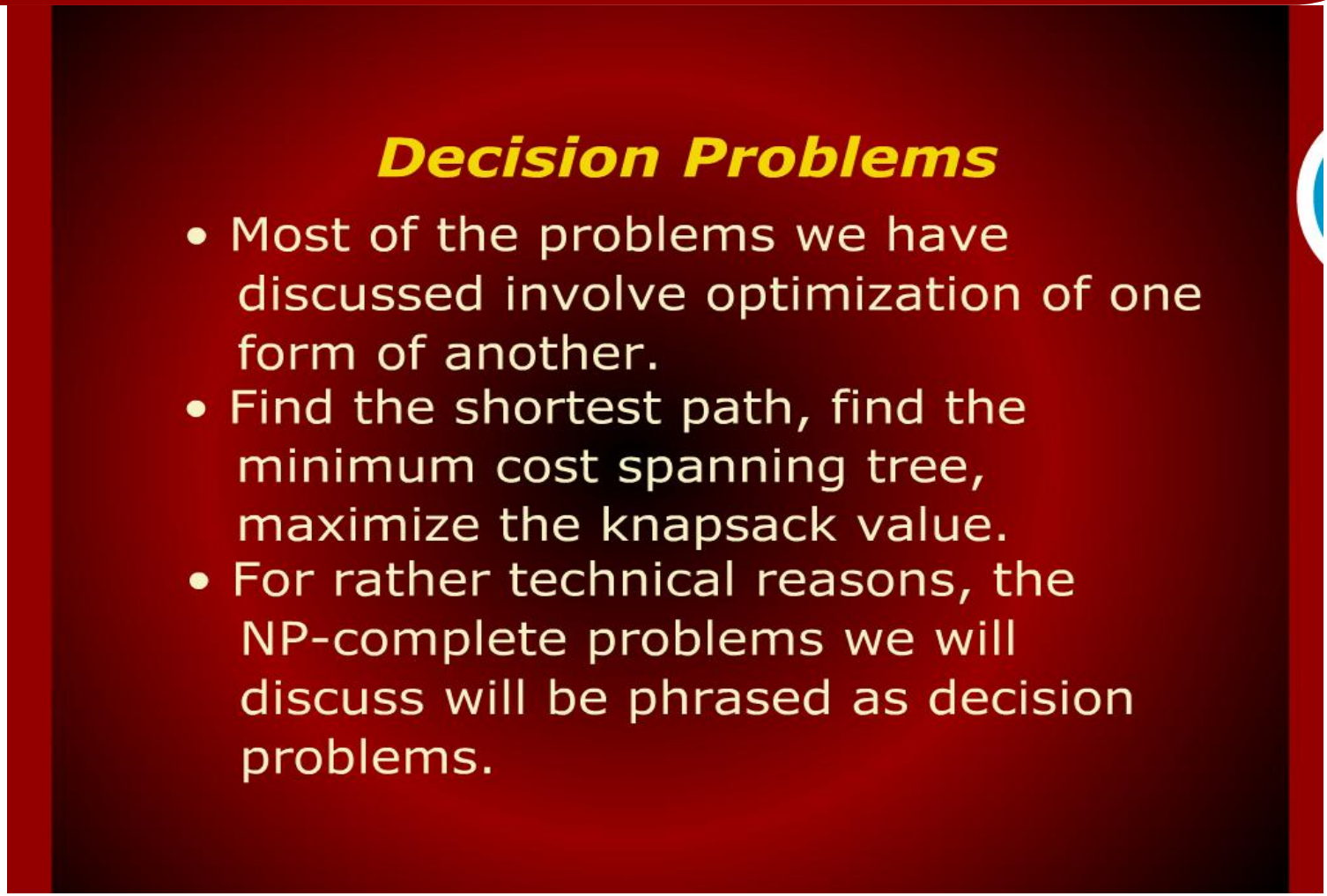
Complexity Theory

- Some functions that do not look like polynomials (such as O(n log n) are bounded above by polynomials (such as $O(n^2)$).
- Some functions that do look like polynomials are not.
- For example, suppose you have an algorithm that takes as input a graph of size n and an integer k and run in $O(n^k)$ time.

Complexity Theory



Decision Problems

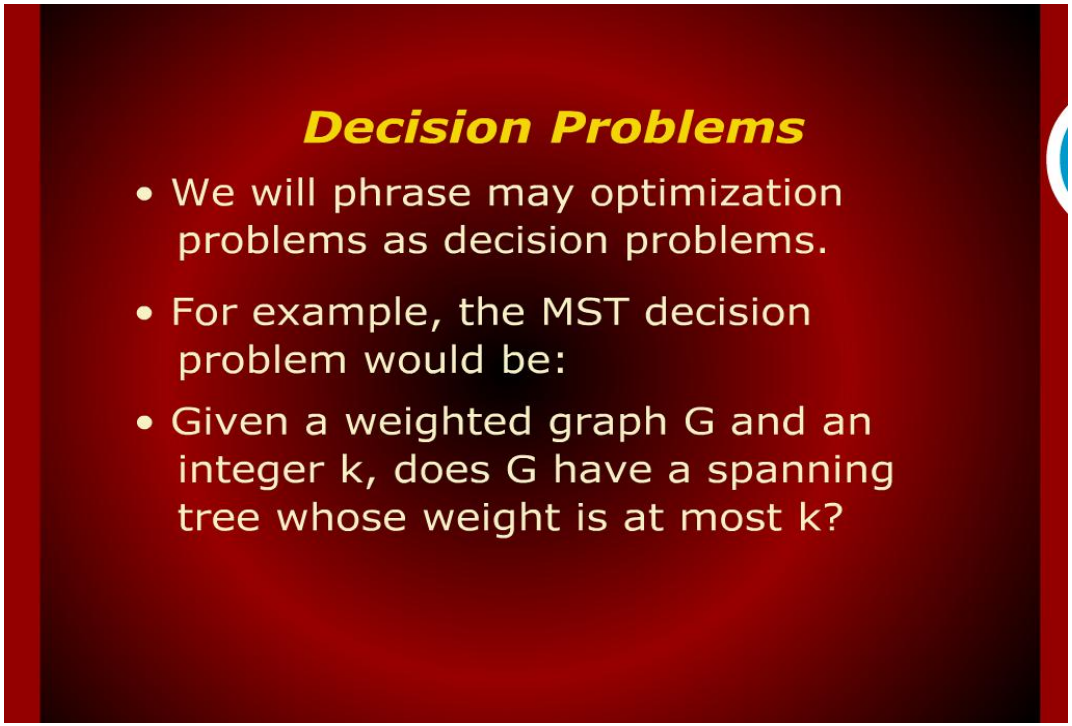


Decision Problems

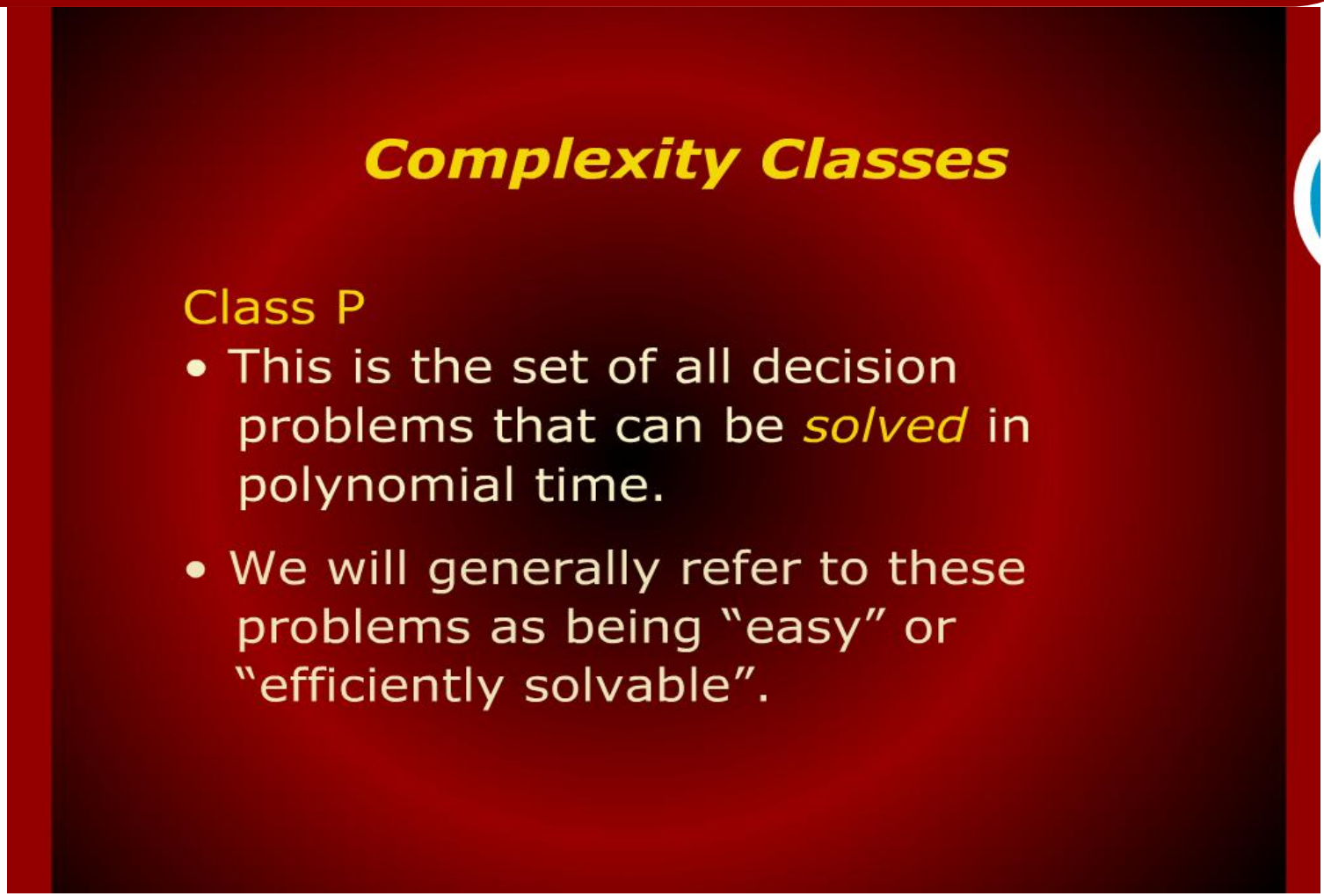
Decision Problems

- A problem is called a *decision problem* if its output is a simple "yes" or "no" (or you may this of this as true/false, 0/1, accept/reject.)
- We will phrase may optimization problems as decision problems.
- For example, the MST decision problem would be:

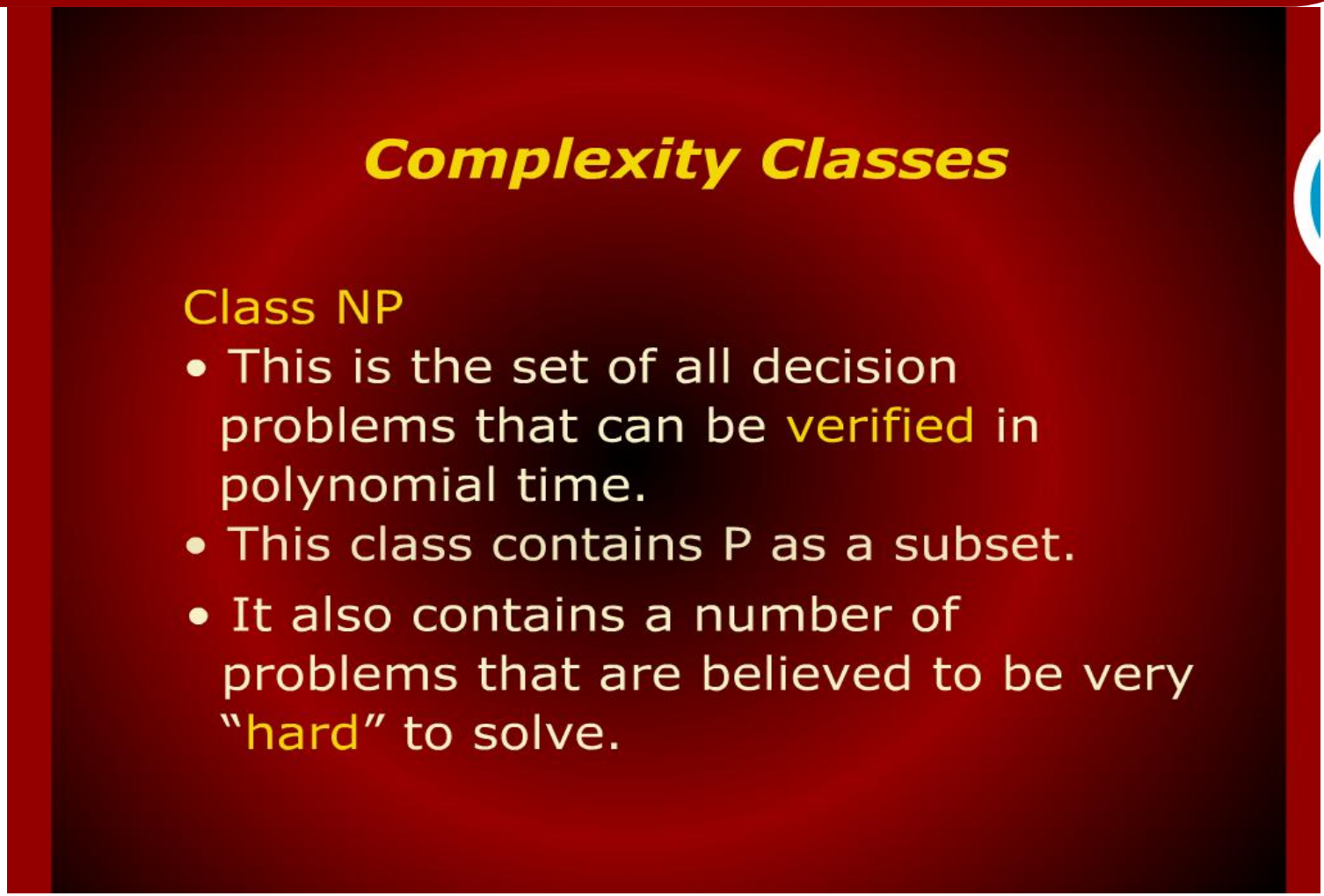
Decision Problems



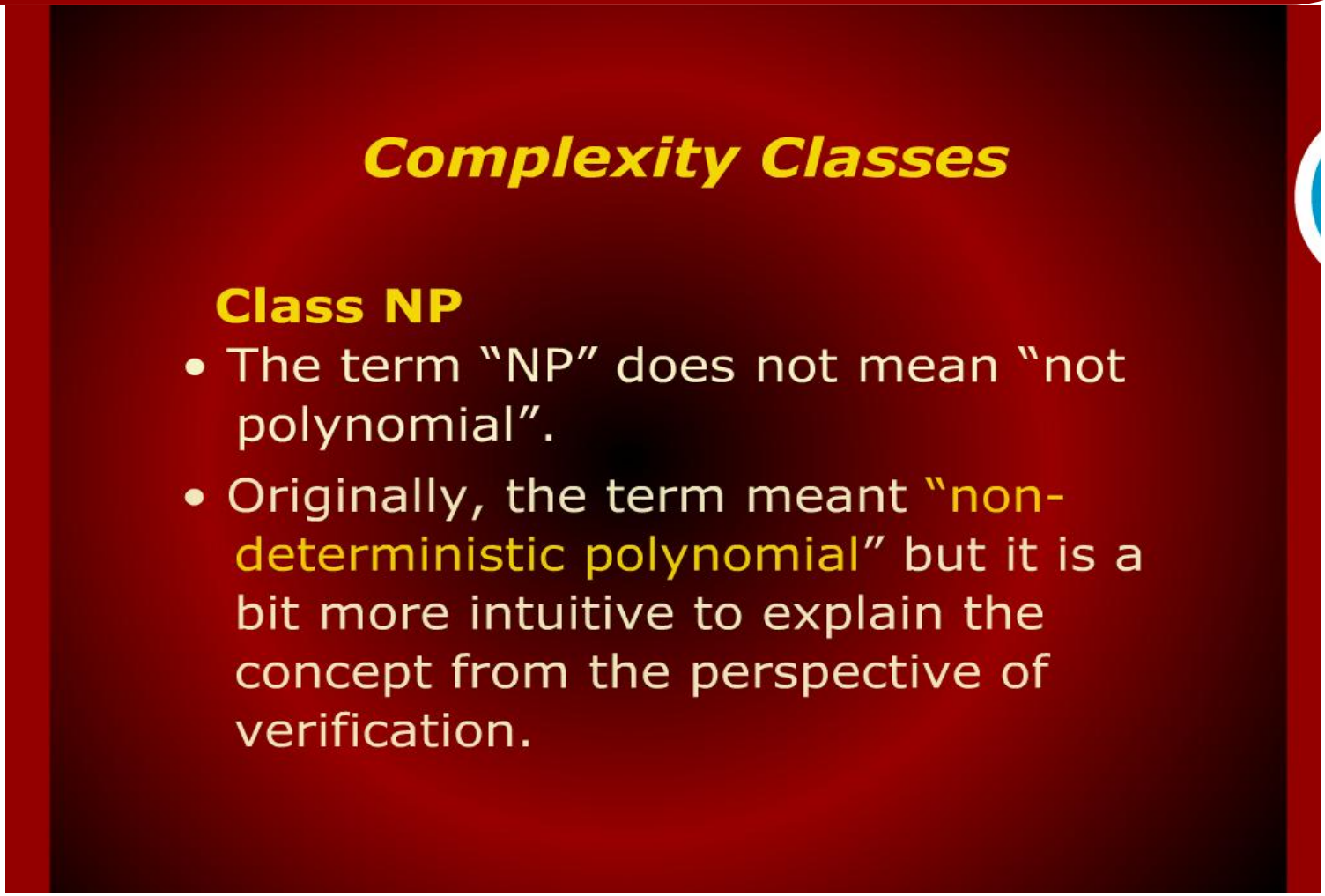
Complexity Classes



Complexity Classes



Complexity Classes



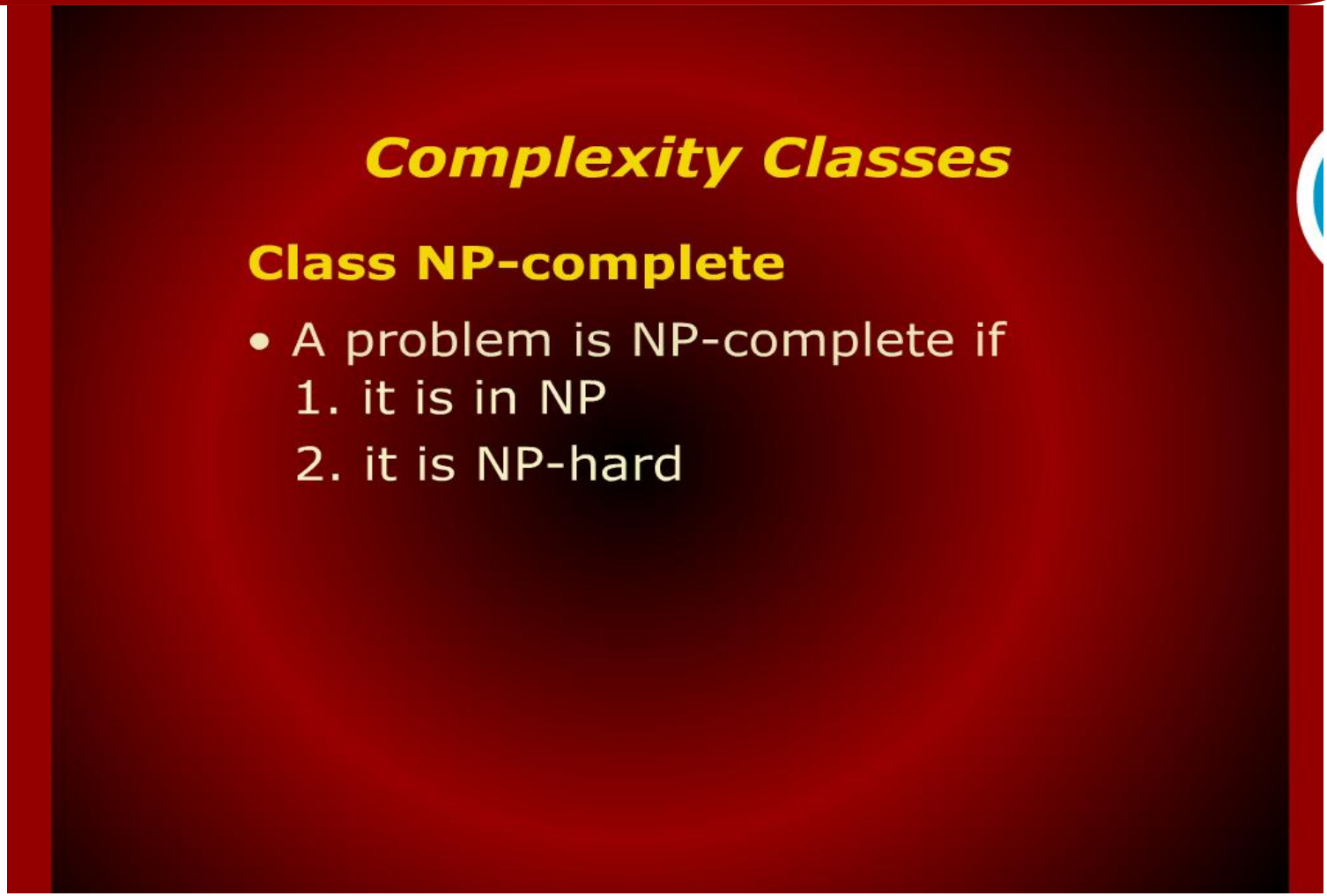
Complexity Classes

Complexity Classes

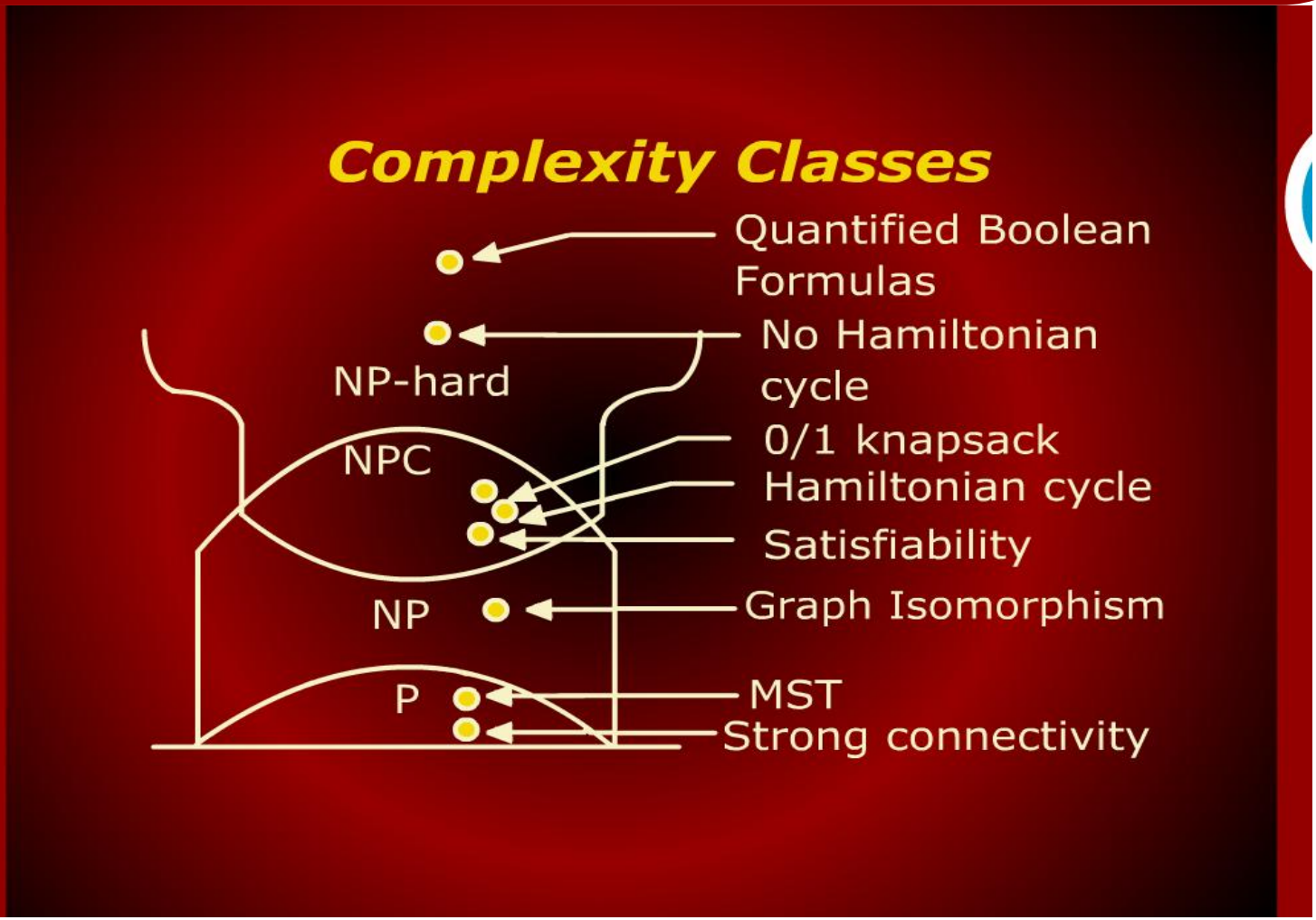
Class NP-hard

- In spite of its name, to say that a problem is NP-hard does not mean that it is hard to solve.
- Rather, it means that if we could solve this problem in polynomial time, then we could solve *all* NP problems in polynomial time.
- Note that for a problem to NP-hard, it does not have to be in the class NP.

Complexity Classes



Complexity Classes

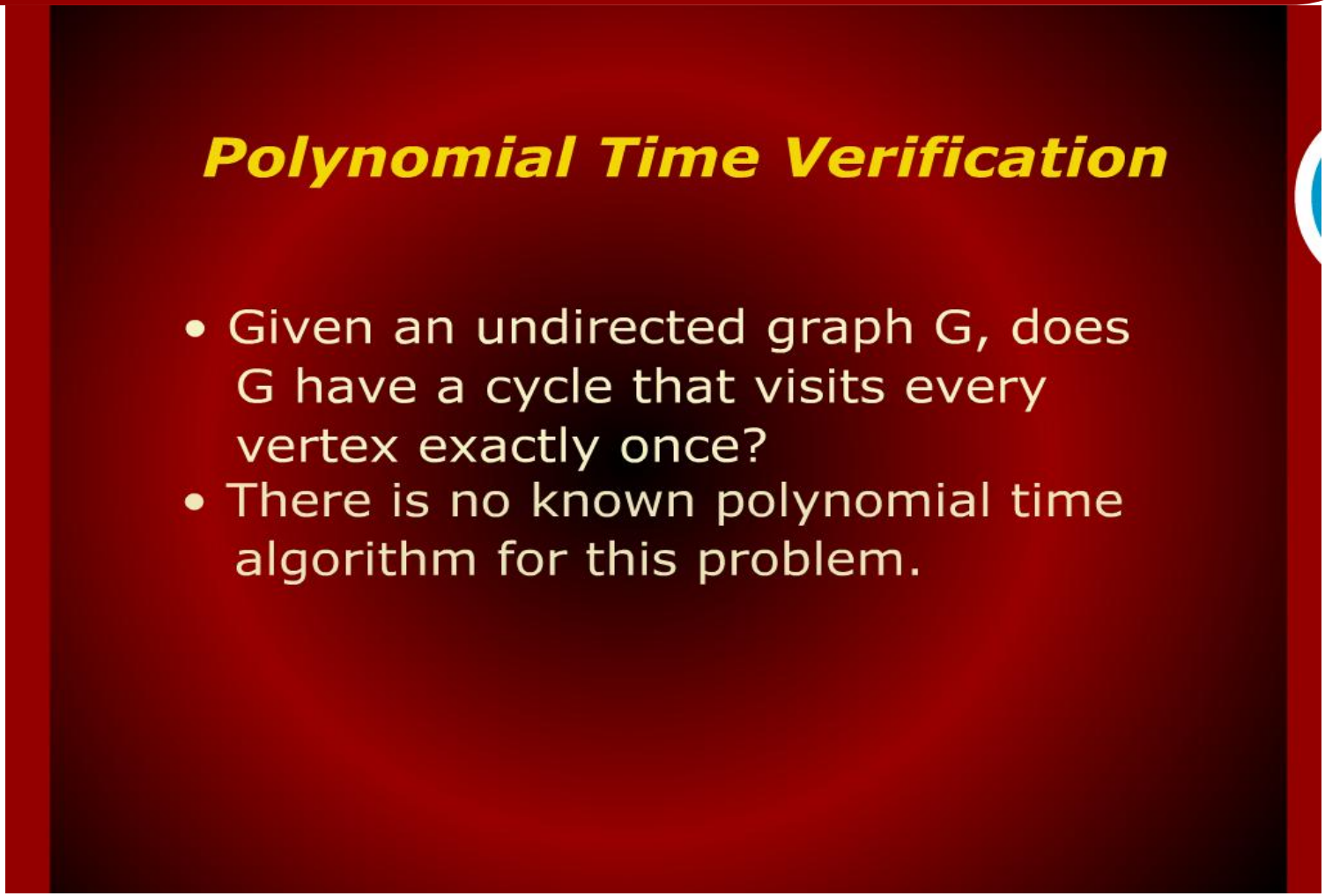


Polynomial Time Verification

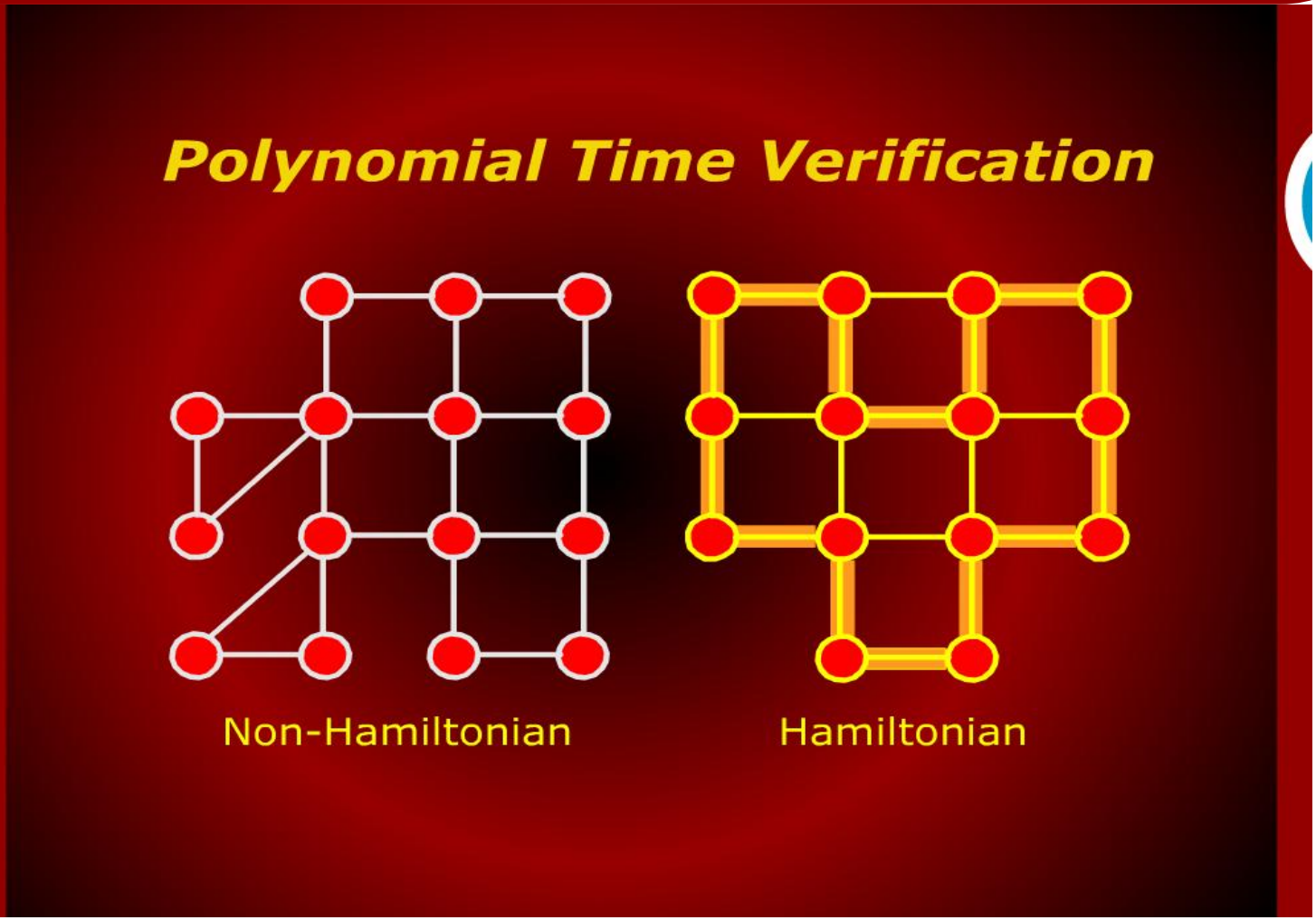
Polynomial Time Verification

- Before talking about the class of NP-complete problems, it is important to introduce the notion of a verification algorithm.
- Many problems are hard to solve but they have the property that it easy to verify the solution if one is provided.
- Consider the Hamiltonian cycle problem.

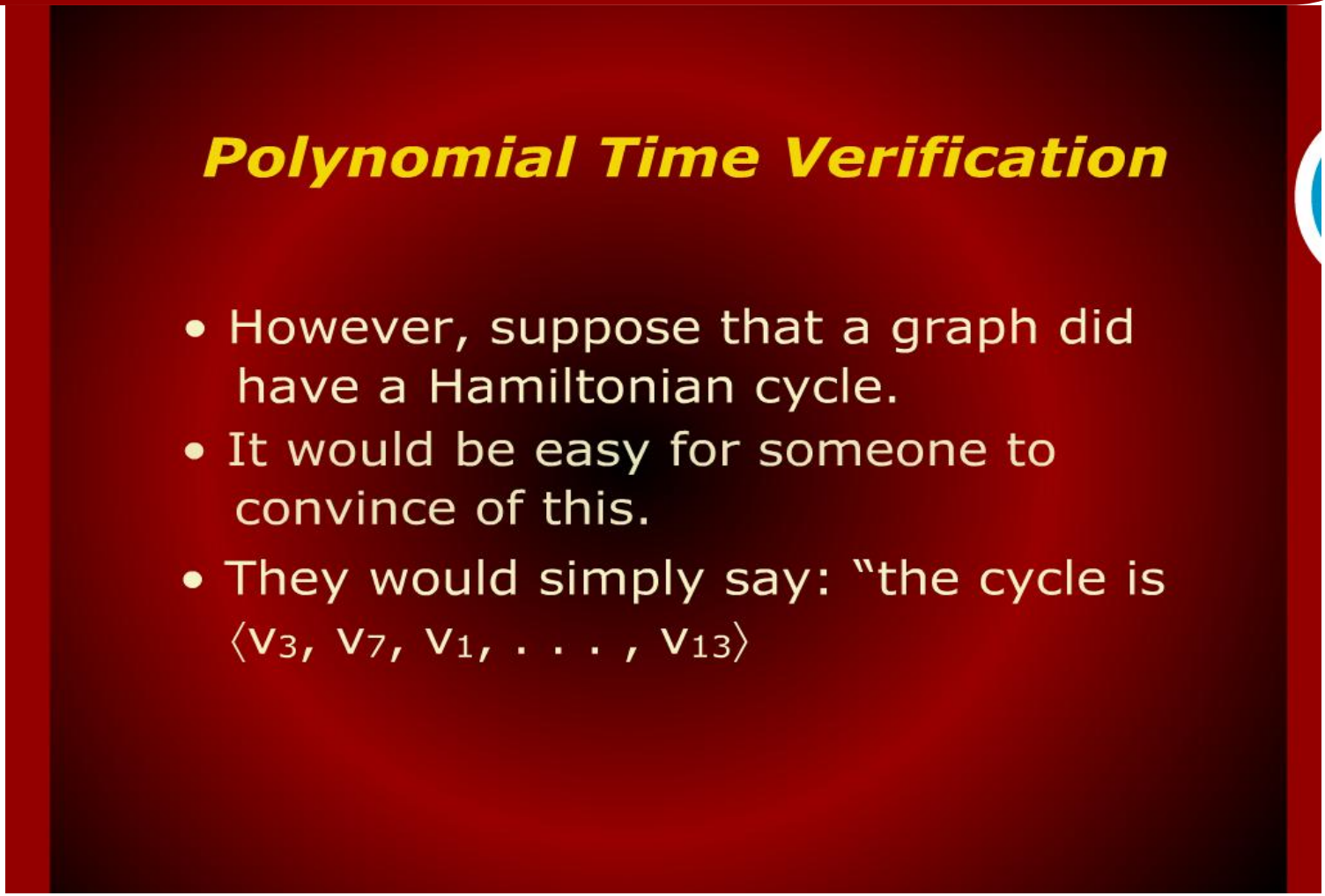
Polynomial Time Verification



Polynomial Time Verification



Polynomial Time Verification



Polynomial Time Verification

Polynomial Time Verification

- We could then inspect the graph and check that this is indeed a legal cycle and that it visits all of the vertices of the graph exactly once.
- Thus, even though we know of no efficient way to *solve* the Hamiltonian cycle problem, there is a very efficient way to *verify* that a a given cycle is indeed a Hamiltonian cycle.

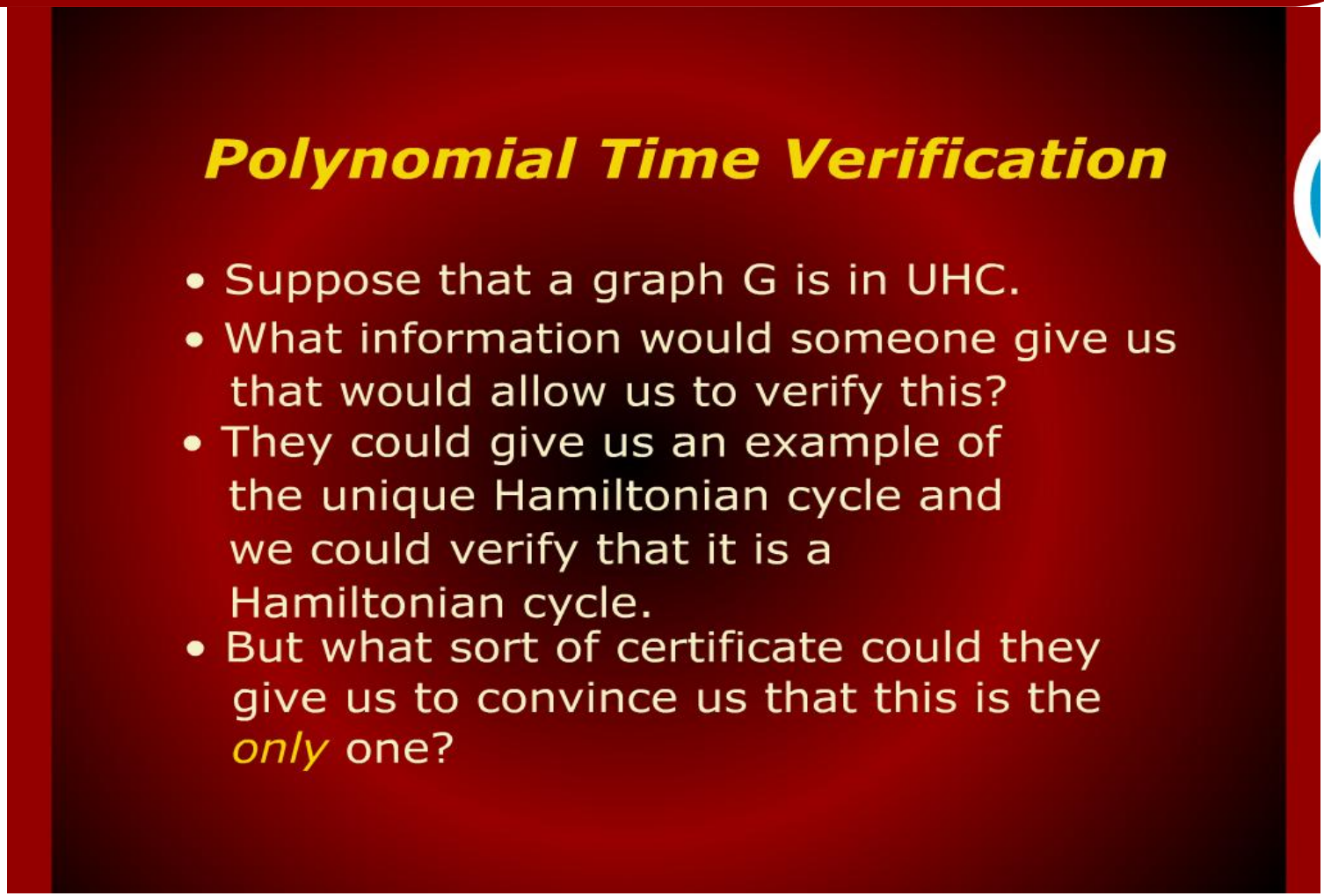
Polynomial Time Verification

Polynomial Time Verification

- The piece of information that allows verification is called a *certificate*.
- Note that not all problems have the property that they are easy to verify.
- For example, consider the following two:

  1. UHC=
     {(G)|G has a unique Hamiltonian cycle}
  2. $\overline{HC}$ =
     {(G)|G has no Hamiltonian cycle}

Polynomial Time Verification

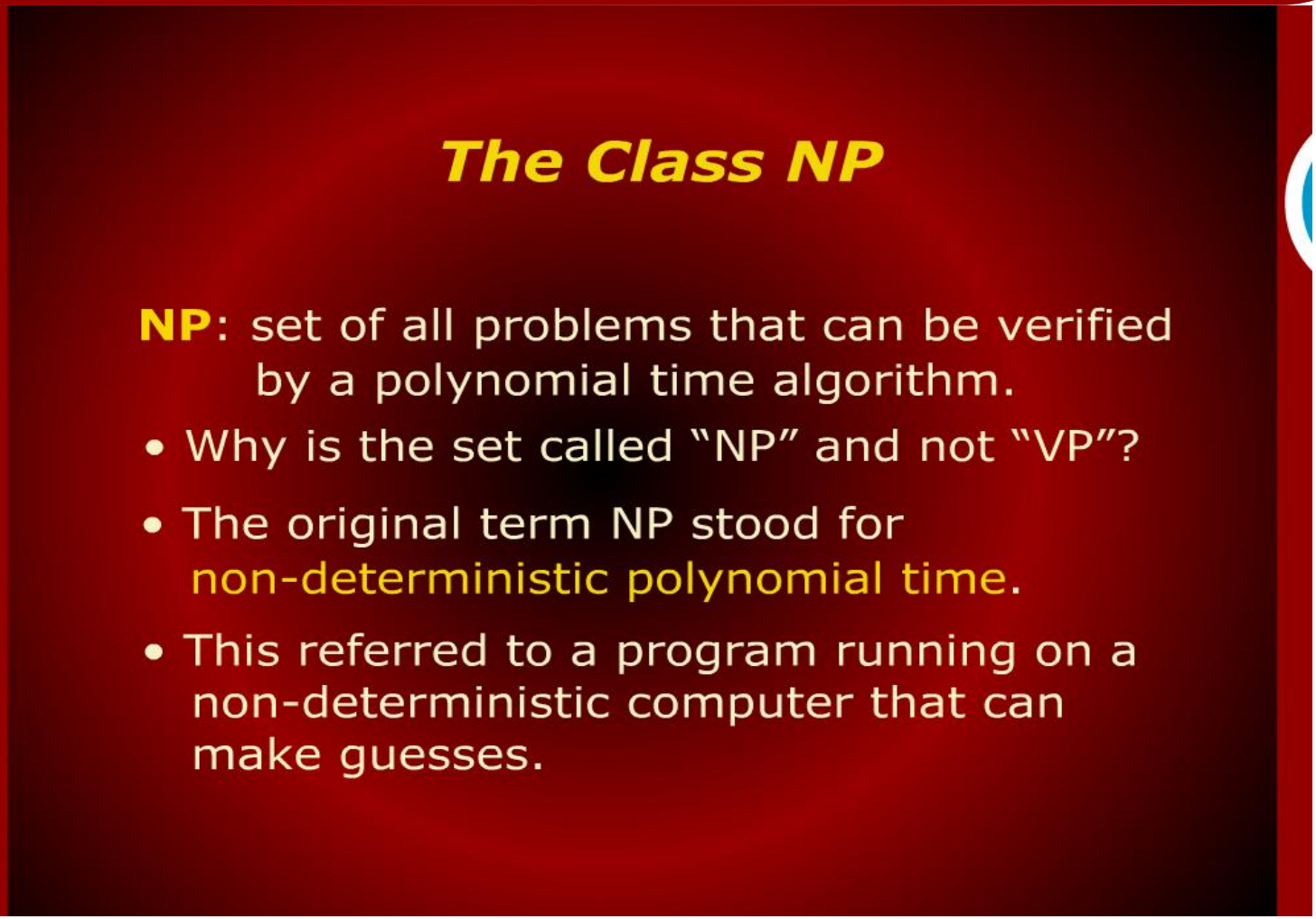


Polynomial Time Verification

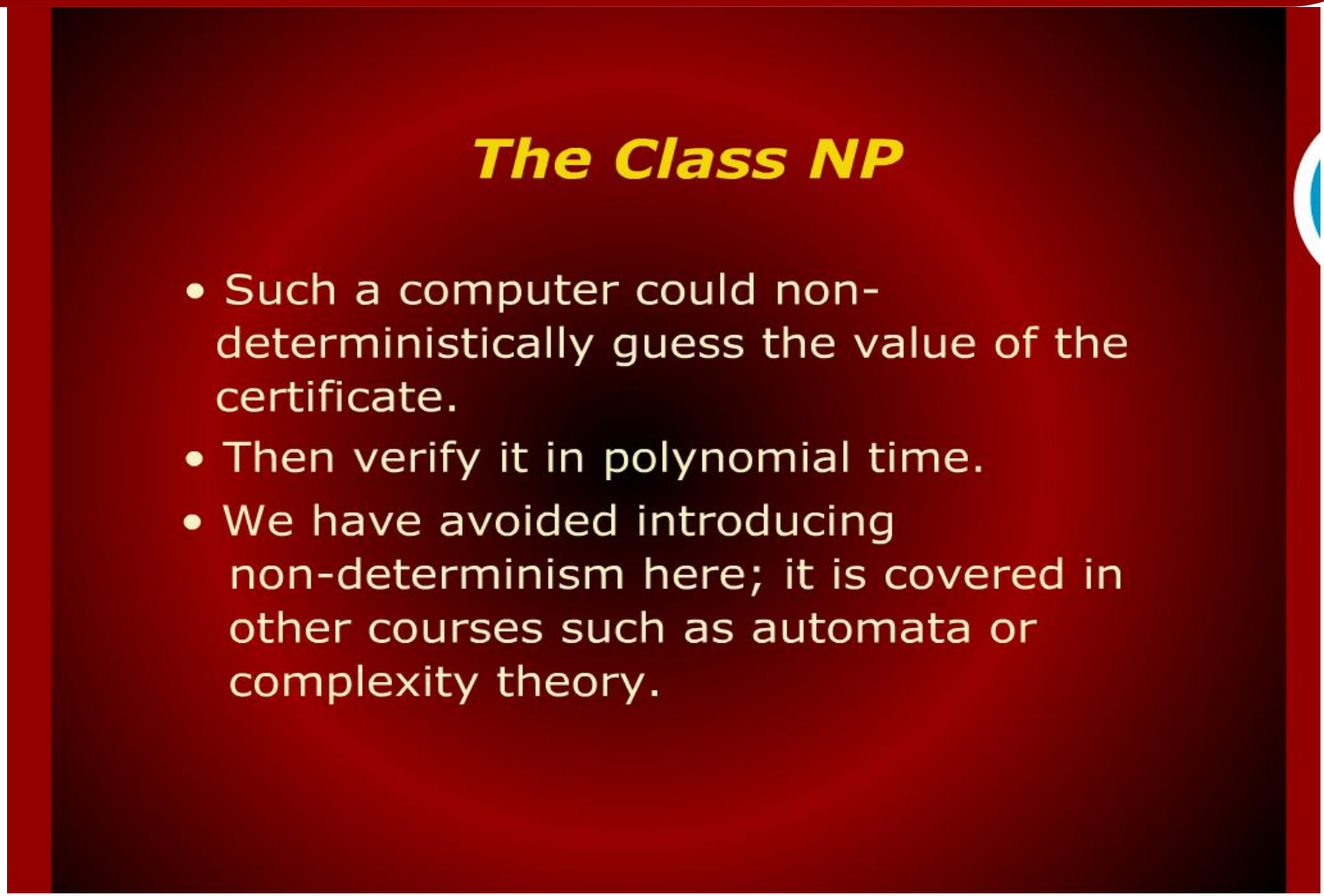
Polynomial Time Verification

- They could give another cycle that is not Hamiltonian.
- But this does not mean that there is not another cycle somewhere that is Hamiltonian.
- They could try to list every other cycle of length n, but this is not efficient at all since there are n! possible cycles in general.

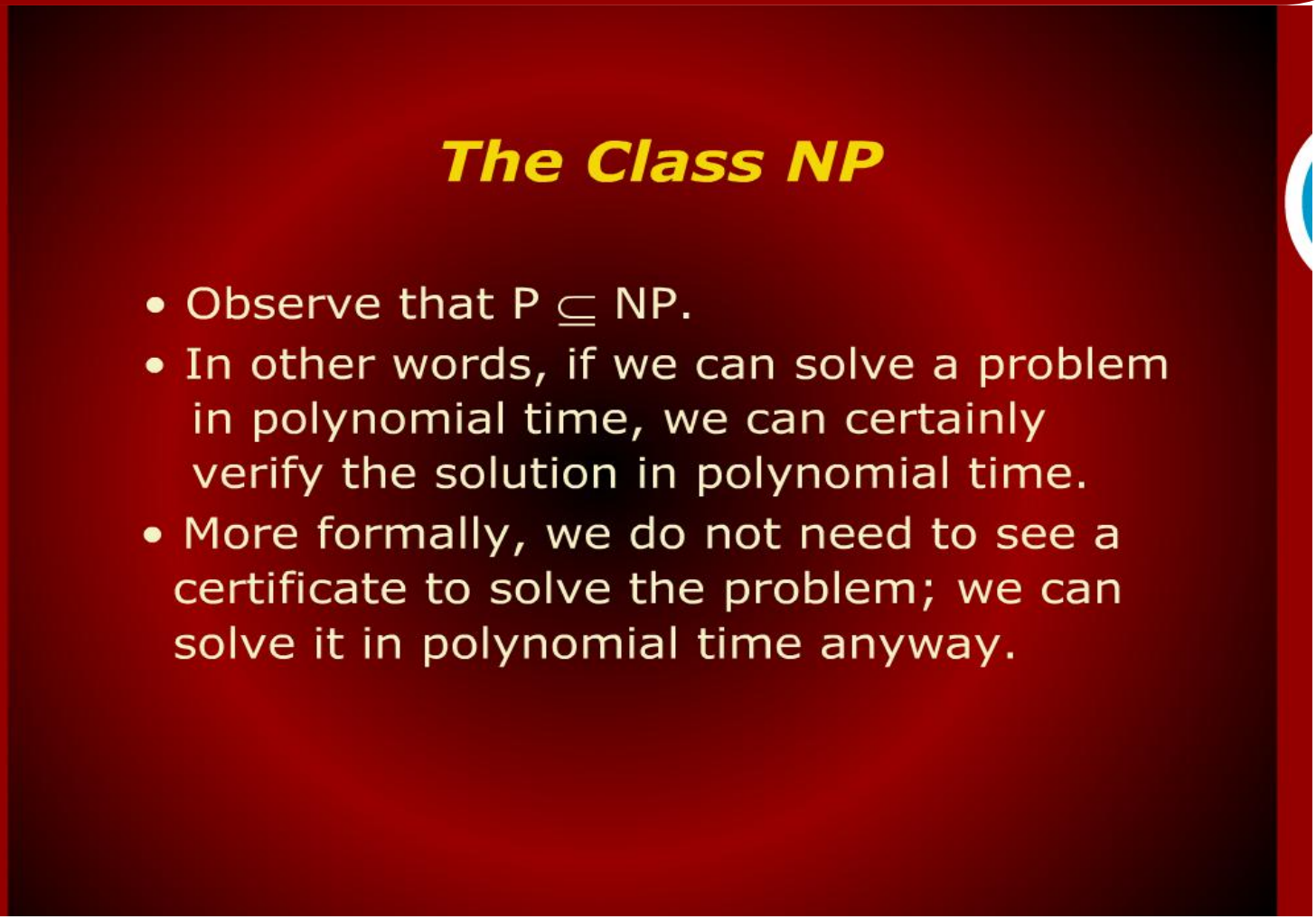
The Class NP


The Class NP
NP: set of all problems that can be verified by a polynomial time algorithm.
• Why is the set called "NP" and not "VP"?
• The original term NP stood for non-deterministic polynomial time.
• This referred to a program running on a non-deterministic computer that can make guesses.

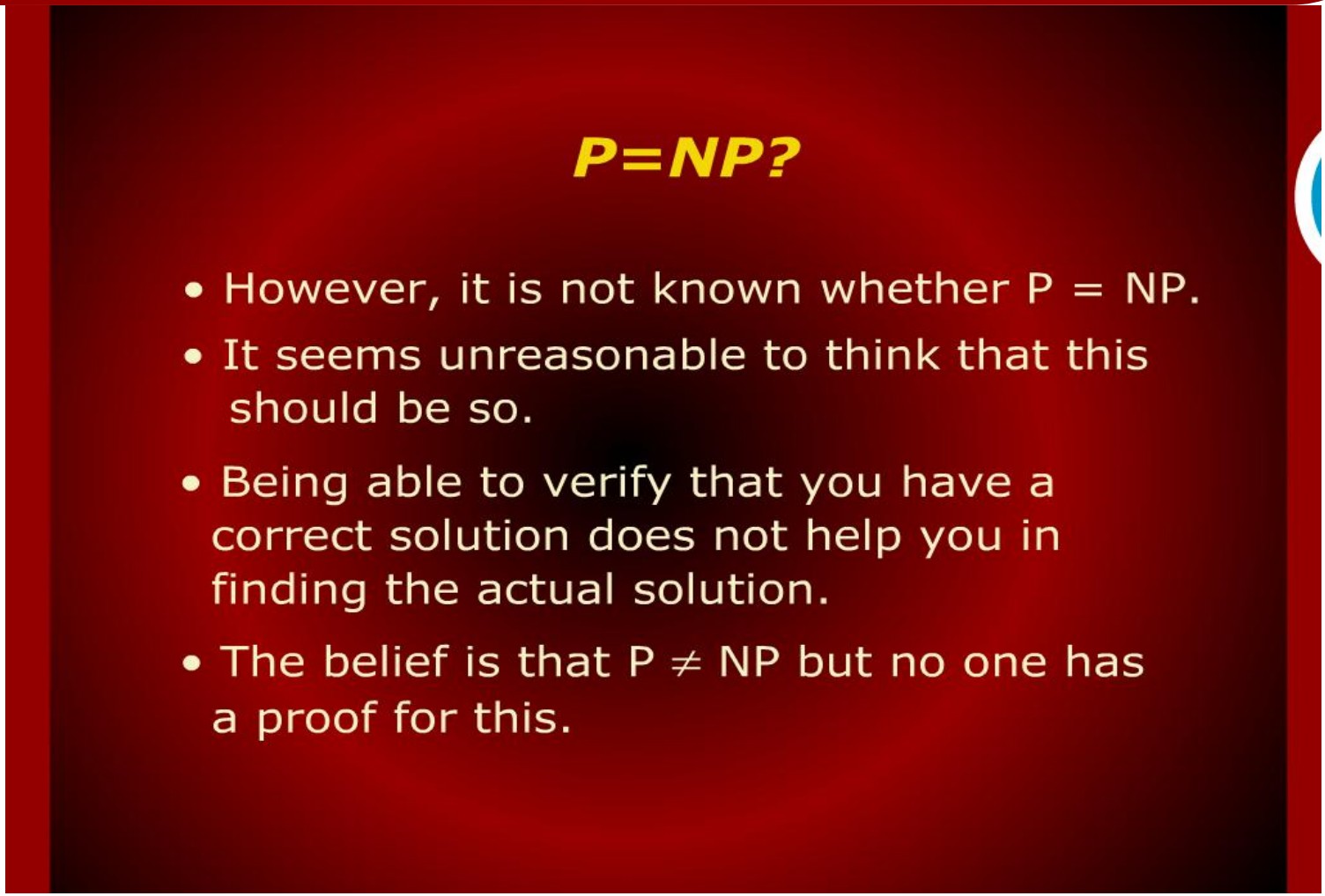
The Class NP


The Class NP
• Such a computer could non-deterministically guess the value of the certificate.
• Then verify it in polynomial time.
• We have avoided introducing non-determinism here; it is covered in other courses such as automata or complexity theory.

The Class NP


The Class NP
• Observe that P ⊆ NP.
• In other words, if we can solve a problem in polynomial time, we can certainly verify the solution in polynomial time.
• More formally, we do not need to see a certificate to solve the problem; we can solve it in polynomial time anyway.

P=NP?


P=NP?
• However, it is not known whether P = NP.
• It seems unreasonable to think that this should be so.
• Being able to verify that you have a correct solution does not help you in finding the actual solution.
• The belief is that P ≠ NP but no one has a proof for this.

Reductions

Reductions

- The class NP-complete problems consists of a set of decision problems (a subset of class NP) that no one knows how to solve efficiently.
- But if there were a polynomial solution for even a single NP-complete problem, then ever problem in NPC will be solvable in polynomial time.
- For this, we need the concept of reductions.